

PITLake

A point-in-time market data lakehouse built on Databricks.

1. What the project is

PITLake collects market and financial data from several public sources, cleans it under enforced quality rules, and stores it so that trading strategies can be tested without ever seeing information that did not exist at the moment they claim to trade.

The deliverable is data infrastructure: ingestion, cleaning, storage, orchestration and monitoring. Finding a profitable strategy is not a goal of this project.

The problem it addresses

Financial datasets routinely misrepresent timing. A company reports fourth-quarter results on 20 February for a quarter that ended on 31 December, but most free datasets label the figure with the December date. A backtest reading it in early January is using information that was not public for another seven weeks. The results look excellent and are worthless. This is called lookahead bias, and it is the most common reason research fails when it reaches production.

Macroeconomic series have the same problem in a different form: published figures are revised months after their first release, so a dataset downloaded today shows numbers nobody could have seen at the time.

PITLake solves this structurally rather than by convention. Every row in the system carries a second timestamp recording when that information became knowable, and the only sanctioned way to read a feature filters on that timestamp.

2. Data sources

Five public sources, all free, covering two asset classes plus fundamentals and macro data. Nothing is simulated.

Source	Data	Access
Binance	Every executed trade on the exchange, published as daily files per trading pair	Public, no key
Coinbase	The same pairs on a second venue, for cross-venue comparison	Public REST API
Alpaca	US equity trades and daily bars via IEX	Free API key
SEC EDGAR	Company financial statements, carrying the actual filing date	Public bulk data
FRED	Interest rates, inflation and unemployment, including revision vintages	Free API key

Having more than one source is not about volume. Two crypto venues quoting the same pair give a free correctness check: if data is missing on one venue and present on the other, that is an ingestion fault rather than a market event. Equities combined with fundamentals and macro data exercise the full point-in-time

join chain, which crypto data alone cannot do.

3. How data enters the platform

Databricks Free Edition runs on serverless compute with outbound internet access restricted to a small set of trusted domains. Calls to external market data APIs therefore cannot be made from inside the workspace. Acquisition runs outside it.

A small collector program runs continuously on a free Linux virtual machine. It calls each source's API on a schedule, verifies the integrity of what it receives, and uploads the untouched result into a Unity Catalog volume inside Databricks. From that point everything happens on the platform.

Properties of the collector

- Runs unattended under systemd, with automatic restart and structured logging.
- Verifies checksums before accepting a file, so corrupt downloads never enter the system.
- Writes atomically, so a partial transfer is never visible to Databricks.
- Respects each source's rate limits and retries with exponential backoff.
- Records what it fetched, when, and from where, in a manifest table inside Databricks.
- Never modifies the data. Raw files are kept permanently and can be reprocessed at any time.

4. Storage layers

Data moves through four layers, each with a defined contract. All are Delta tables governed by Unity Catalog.

Layer	Contents
Raw	The original files exactly as received. Never edited. The audit trail and the source of any reprocessing.
Bronze	The same records loaded into tables without transformation. Malformed columns are preserved rather than causing a failure.
Silver	Cleaned and typed. Duplicates removed, timestamps normalised, symbols reconciled across venues, quality rules enforced, and the knowable-at timestamp assigned.
Gold	Aggregated and research-ready. Trades compressed into bars, features computed, fundamentals and macro data joined on their true publication dates.

A separate control area holds operational state: what has been ingested so far, quality thresholds, the registry of active sources, and cost records.

5. Data quality

Cleaning rules are declared in the pipeline definition rather than scattered through processing code. Each rule either drops the offending row or halts the run.

Rule	On failure	Purpose
Price and quantity positive	Drop row	Removes impossible trades
Knowable-at timestamp present	Halt run	A missing value here would silently disable point-in-time protection
Knowable-at not before the event	Halt run	Catches a mis-assigned timestamp
Timestamp within plausible range	Drop row	Catches unit errors, where one source reports milliseconds and another microseconds
Source is registered	Halt run	Prevents unreviewed data entering Silver
Sequence numbers continuous	Record only	A gap is a fact about the feed and is kept, not discarded

Rejected rows are not deleted. They are written to a separate table together with the name of the rule that rejected them, so rejection rates can be measured, explained and defended.

6. Point-in-time correctness

Every record receives a timestamp recording when its information genuinely became obtainable. It is assigned once, during cleaning, and never recalculated downstream.

Data	Knowable at
Trades	The exchange timestamp of the trade
Company fundamentals	The filing date reported by the SEC, never the end of the reporting period
Restated fundamentals	The filing date of the amendment, retained as a new record rather than overwriting the original
Macroeconomic series	The release date of that particular vintage, not the date of the latest revision

Features are reachable only through a shared function that requires a timestamp and returns the most recent value knowable at that instant. A test suite deliberately constructs data whose knowable-at timestamp is later than the event and asserts that it never appears in any result. This suite runs on every change to the codebase.

7. Automation

Everything inside Databricks runs as scheduled jobs composed of dependent tasks. No step is run by hand.

Job	Schedule	Tasks
Ingest	Daily	Determine which data is missing, load new files from each source in parallel, then advance the record of what has been processed only if every source succeeded

Job	Schedule	Tasks
Transform	Hourly	Run the cleaning and aggregation pipeline, compare the two crypto venues against each other, then stop the run if the rejection rate exceeds its threshold
Research	On demand	Record the exact version of every table to be read, build features, run backtests across parameter sets, and log every result
Operations	Daily	Compact and optimise storage, apply retention, and report compute cost per job from the platform's own usage tables

Tasks that process many symbols fan out into parallel iterations with a concurrency limit set to respect each source's rate limits. Failed tasks retry automatically; a task that fails after its retries stops the job and sends an alert rather than allowing incomplete data to reach the research layer.

8. Application and monitoring

A Databricks App provides the interface to the platform. It shows which data is present for which sources and dates, current rejection rates by rule, disagreements found between the two crypto venues, and a browser for past backtest runs. It also allows the ingestion job to be triggered for a specific source and date range.

Separately, automated monitoring watches the Silver tables for staleness and for changes in data distribution, so that a change to an upstream source's format is detected on the day it happens rather than weeks later.

9. Databricks components used

Component	Role in the project
Unity Catalog	Governs all data. Separate development and production catalogs, automatic lineage from any table back to its source file
Volumes	Hold the raw uploaded files
Auto Loader	Detects and loads new files incrementally without duplicate processing
Declarative pipelines	Define the cleaning and aggregation layers, including the quality rules and the quarantine of rejected rows
Delta Lake	Table format for every layer. Guarantees that a failed job never leaves a partially written table
Delta version history	Every table retains a version number for every write. This is what makes a result reproducible
Liquid clustering	Physically organises the Gold tables by symbol and time to reduce query cost
Jobs	Orchestrate all four workflows, with task dependencies, retries and alerting
Asset Bundles	Define the entire platform as version-controlled configuration, deployed automatically rather than configured by hand

Component	Role in the project
Feature tables	Store computed features with point-in-time access semantics
MLflow	Records every backtest with its parameters, results, code version and the table versions it read
SQL dashboards and alerts	Operational visibility and notification when quality thresholds are breached
Lakehouse Monitoring	Detects staleness and distribution changes in cleaned data
Apps	Hosts the console described above
System tables	Provide the platform's own job and cost history for reporting

10. Stages

Seven stages, ordered by dependency rather than by date. Each ends with a test that either passes or does not; a stage is not complete because work was done on it.

Stage	Scope	Complete when
0	Repository, deployment configuration, catalogs, credentials, automated testing	An empty deployment reaches the workspace from the code repository in one command
1	Collector for the first source, raw upload, loading into Bronze	One year of one symbol is loaded, and reloading the same period adds no duplicate rows
2	Remaining four sources behind a common interface	All five sources load through the same job with no source-specific handling in the job itself
3	Cleaning layer, quality rules, quarantine, knowable-at timestamps	Rejection rates are measured and every category explained; the timestamp rules are proven to halt the run on deliberately corrupted input
4	Cross-venue comparison and the quality gate	An injected discrepancy stops the job instead of reaching the research layer
5	Aggregation, features, point-in-time access, leakage testing	The leakage test suite passes zero cases, and a company's results are provably invisible before their filing date
6	Research workflow, experiment tracking, backtesting	A backtest reproduced from its recorded versions returns identical results
7	Operations, application, monitoring, performance work, documentation	Query performance improvements are measured and published, and another person can deploy and run the system from the documentation alone

Stages 1 and 2 can overlap. Everything from stage 3 onward is sequential.

Timeline

Stage	Dates	Weeks
0	17 September to 4 October 2026	2.5
1	5 October to 15 November 2026	6
2	16 November to 20 December 2026	5
—	21 December 2026 to 3 January 2027, no work scheduled	2
3	4 January to 21 February 2027	7
4	22 February to 21 March 2027	4
5	22 March to 16 May 2027	8
6	17 May to 4 July 2027	7
7	5 July to 12 September 2027	10

Fifty weeks of scheduled work across twelve months, with a two-week break at the end of December. The estimates assume part-time effort alongside coursework. Stages 3 and 5 carry the largest allowances because they contain the parts of the system most likely to require rework: the quality rules and the point-in-time guarantees. If a stage overruns, the time is taken from stage 7, which is the only stage whose scope can be reduced without weakening what comes before it.

11. Scope

Included

- Five data sources, a fixed and deliberately small universe of instruments.
- Ingestion, cleaning, storage, aggregation and feature computation.
- Point-in-time guarantees and reproducibility of every result.
- Full automation, monitoring and cost reporting.

Excluded

- Live or paper trading of any kind.
- Real-time streaming ingestion, which the free tier cannot support.
- Searching for profitable strategies. The backtester exists to demonstrate reproducibility, not to generate returns.
- Paid data vendors, options and derivatives.

12. Constraints and risks

Constraint or risk	Response
No outbound internet access from the workspace	Acquisition runs on an external free virtual machine, which is also what gives the project a systems engineering component
Daily compute quota; exceeding it suspends compute	Historical loading is divided into small chunks with recorded progress, so an interrupted run resumes rather than restarting
Limited pipeline and warehouse capacity	A single pipeline handles all cleaning; the application and dashboards read only the aggregated layer, never raw data
A source changes its format or symbols	Monitoring and the source registry rule surface the change on the next run; each source is isolated in its own module, so the fix is local
Available time is reduced by coursework and internship	Every stage is independently demonstrable. Stopping after stage 6 still yields a complete and coherent system
Scope drifts toward strategy research	Profitability is an explicit non-goal, restated in the scope section above

Project specification, revision 3.